Mathematics for Deep Learning

And other abstract nonsense

aNLP M'26
August 7, 2026

IIIT Hyderabad

“““““Prereqs”””””

- Basic Linear Algebra (an idea of what a vector is and what a matrix is)
- Basic Calculus (derivatives)
- Barebones intuition for these two (alternatively, a 3b1b subscription)

Linear Algebra Foundations

Vectors, Vector Spaces, and Bases

- **Vector Space** $(V, +, \cdot)$: A set of elements closed under addition and scalar multiplication over a field F (typically $\mathbb{R}$).
- **Basis**: A minimal set of linearly independent vectors $\{b_1, b_2, \dots, b_n\}$ that span V.
- **Linear Combination**: Every vector $v \in V$ is uniquely represented as:

$$v = c_1 b_1 + c_2 b_2 + \cdots + c_n b_n$$

- **Span**: The set of all possible linear combinations of a set of vectors.

Matrices as Linear Transformations

- **Matrix as a Map**: $A \in \mathbb{R}^{m \times n}$ defines a linear transformation $T : \mathbb{R}^n \to \mathbb{R}^m$:

$$A_{m \times n} x_{n \times 1} = y_{m \times 1}$$

- **Where Basis Vectors Land**: The columns of A state explicitly where the standard basis vectors land after transformation.
- **Geometric Properties**:
  - Grid lines remain straight and parallel.
  - Origin remains fixed: $T(0) = 0$.

Primitive Spatial Transformations

- **Diagonal Matrices** ($D = \text{diag}(d_1, \dots, d_n)$): Pure scaling along standard coordinate axes without rotation.
- **Orthogonal Matrices** ($Q^T Q = Q Q^T = I$): Pure rotation or reflection. Preserves lengths and angles ($\det(Q) = \pm 1$).

Eigenvalues & Eigenvectors

- **Invariant Directions**: Non-zero vectors v that do not change direction under transformation A, only magnitude:

$$Av = \lambda v \iff (A - \lambda I)v = 0$$

- **Characteristic Equation**:

$$\det(A - \lambda I) = 0$$

Low-Rank Matrices via Outer Products

Rank-1 Matrix (Outer Product)

For vectors $u \in \mathbb{R}^d$ and $v \in \mathbb{R}^k$, the outer product $uv^T \in \mathbb{R}^{d \times k}$ has **rank 1**.

Rank-r Matrix Factorization

Summing r rank-1 matrices yields a low-rank matrix that can be grouped into two thin matrices B and A:

$$\Delta W = \sum_{i=1}^{r} u_i v_i^T = BA$$

where $B \in \mathbb{R}^{d \times r}$ and $A \in \mathbb{R}^{r \times k}$ with rank $r \ll \min(d, k)$.

- **Parameter Economy**: Storing B and A requires $r(d + k)$ parameters instead of full $d \cdot k$.

The LLM Fine-Tuning Bottleneck

- **Full Fine-Tuning**: Pre-trained weight $W_0 \in \mathbb{R}^{d \times k}$ is updated as $W = W_0 + \Delta W$.
- **Parameter Explosion**: Updating dense ΔW directly requires optimizing all $d \cdot k$ parameters per matrix.
- **Intrinsic Rank Hypothesis**: Weight updates ΔW during task adaptation have a very low **intrinsic rank**. $\implies$ The essential learning occurs in a low-dimensional subspace!

Low-Rank Adaptation (LoRA)

LoRA Formulation

Explicitly parameterize the update matrix ΔW as the product of two trainable low-rank matrices:

$$W = W_0 + \Delta W = W_0 + BA$$

where $B \in \mathbb{R}^{d \times r}$ and $A \in \mathbb{R}^{r \times k}$ with rank $r \ll \min(d, k)$.

- **Frozen Base**: Pre-trained weights W_0 remain completely **frozen**.
- **Trainable Adapters**: Only low-rank matrices A and B are updated via gradient descent.
- **Parameter Reduction**: Drops parameter footprint from $d \cdot k$ down to $r(d + k)$ (often $> 99\%$ savings).

Theoretical Foundation: Why Does This Work?

SVD (Eckart–Young Theorem) proves that any matrix update can be optimally approximated by a sum of top-r rank-1 components without significant information loss—giving us theoretical permission to model ΔW as a low-rank matrix BA.

Calculus

Why would you differentiate matrices?

First principles?

$$\lim_{\Delta x \to 0} \frac{f(x + \Delta x) - f(x)}{\Delta x}$$

Why would you differentiate matrices?

First principles?

$$\lim_{\Delta x \to 0} \frac{f(x + \Delta x) - f(x)}{\Delta x}$$

Fair enough

$$\frac{d}{dx}\left[a_{ij}(x)\right] = \left[\frac{da_{ij}}{dx}(x)\right]$$

Why are there so many variables?

- How would you differentiate a multivariable function?

Why are there so many variables?

- How would you differentiate a multivariable function?
- $f(x, y) = x^2 + y^2 + 2xy$

Why are there so many variables?

- How would you differentiate a multivariable function?
- $f(x, y) = x^2 + y^2 + 2xy$
- Naïve intuition tells us $\frac{df}{dx} = 2x + 2y$, 12th grade math tells us $\frac{df}{dx} = (2x + 2y)\left(1 + \frac{dy}{dx}\right)$

Why are there so many variables?

- How would you differentiate a multivariable function?
- $f(x, y) = x^2 + y^2 + 2xy$
- Naïve intuition tells us $\frac{df}{dx} = 2x + 2y$, 12th grade math tells us $\frac{df}{dx} = (2x + 2y)\left(1 + \frac{dy}{dx}\right)$
- There is a way to go about doing the intuitive, and that's partial derivatives.

Why are there so many variables?

- How would you differentiate a multivariable function?
- $f(x, y) = x^2 + y^2 + 2xy$
- Naïve intuition tells us $\frac{df}{dx} = 2x + 2y$, 12th grade math tells us $\frac{df}{dx} = (2x + 2y)\left(1 + \frac{dy}{dx}\right)$
- There is a way to go about doing the intuitive, and that's partial derivatives.
- $\frac{\partial f}{\partial x}$ can be thought of as the derivative when every other variable is a constant.

Dear Lord, it's Physics!

- When computing the derivative of a multivariable scalar-valued function, it makes sense to consider ”components” of its derivative along the coordinate axes.

Dear Lord, it's Physics!

- When computing the derivative of a multivariable scalar-valued function, it makes sense to consider "components" of its derivative along the coordinate axes.
- Each component must be $\frac{\partial f}{\partial x_i}$.

Dear Lord, it's Physics!

- When computing the derivative of a multivariable scalar-valued function, it makes sense to consider "components" of its derivative along the coordinate axes.
- Each component must be $\frac{\partial f}{\partial x_i}$.
- Et voilá! $\nabla f = \left[\frac{\partial f}{\partial x_i}\right]^\top$ (or $\sum_{i=0}^{n} \frac{\partial f}{\partial x_i}\widehat{x_i}$ for you filthy physicists)

Corn on the Jacob

- For $f : \mathbb{R}^n \to \mathbb{R}^m$, how would you deal with $\frac{df}{dx}$? What does that even mean?

Corn on the Jacob

- For $f : \mathbb{R}^n \to \mathbb{R}^m$, how would you deal with $\frac{df}{dx}$? What does that even mean?
- You'll have to deal with derivatives of $[y_1, y_2, \dots, y_m]$ w.r.t $[x_1, x_2, \dots, x_n]$.

Corn on the Jacob

- For $f : \mathbb{R}^n \to \mathbb{R}^m$, how would you deal with $\frac{df}{dx}$? What does that even mean?
- You'll have to deal with derivatives of $[y_1, y_2, \dots, y_m]$ w.r.t $[x_1, x_2, \dots, x_n]$.
- How'd you deal with the mn possible values of $\frac{\partial y_i}{\partial x_j}$?

Corn on the Jacob

- For $f : \mathbb{R}^n \to \mathbb{R}^m$, how would you deal with $\frac{df}{dx}$? What does that even mean?
- You'll have to deal with derivatives of $[y_1, y_2, \dots, y_m]$ w.r.t $[x_1, x_2, \dots, x_n]$.
- How'd you deal with the mn possible values of $\frac{\partial y_i}{\partial x_j}$?
- $\mathbf{J} = \begin{bmatrix} \nabla^\top y_1 \\ \nabla^\top y_2 \\ \vdots \\ \nabla^\top y_m \end{bmatrix} = [\frac{\partial y_i}{\partial x_j}]$ is how!

We've had one breakfast, yes...

- What about second derivatives? It does make sense to want second derivatives since they're useful for detecting the locations of extrema (and for one other thing that we will see later).

We've had one breakfast, yes...

- What about second derivatives? It does make sense to want second derivatives since they're useful for detecting the locations of extrema (and for one other thing that we will see later).
- For $f : \mathbb{R}^n \to \mathbb{R}$, we already saw the first derivative being $\nabla f : \mathbb{R}^n \to \mathbb{R}^n$.

We've had one breakfast, yes...

- What about second derivatives? It does make sense to want second derivatives since they're useful for detecting the locations of extrema (and for one other thing that we will see later).
- For $f : \mathbb{R}^n \to \mathbb{R}$, we already saw the first derivative being $\nabla f : \mathbb{R}^n \to \mathbb{R}^n$.
- Since this is a vector, we can take the Jacobian of this, which gives $\mathbf{J}\nabla f = [\frac{\partial^2 f}{\partial x_j \partial x_i}]$.

We've had one breakfast, yes...

- What about second derivatives? It does make sense to want second derivatives since they're useful for detecting the locations of extrema (and for one other thing that we will see later).
- For $f : \mathbb{R}^n \to \mathbb{R}$, we already saw the first derivative being $\nabla f : \mathbb{R}^n \to \mathbb{R}^n$.
- Since this is a vector, we can take the Jacobian of this, which gives $\mathbf{J}\nabla f = [\frac{\partial^2 f}{\partial x_j \partial x_i}]$.
- Convention and convenience dictates that we use the transpose $\mathbf{H} = (\mathbf{J}\nabla f)^\top$

It’s all downhill from here (Gradient Descent)

Consider a situation where you’re on a hill and want to reach the lowest point. A local minima will do (for now). How would you go about it if the only thing you could see is the spot where you are and the properties of the land there?

High-concept stuff right here (Newton-Raphson)

- Given a function $f : \mathbb{R} \to \mathbb{R}$, we seek to iteratively compute the minima.

High-concept stuff right here (Newton-Raphson)

- Given a function $f : \mathbb{R} \to \mathbb{R}$, we seek to iteratively compute the minima.
- Essentially, we want to construct a sequence $\{x_i \mid i \in \mathbb{N}\}$ which converges to $\operatorname{argmin}_{x \in \mathbb{R}} (f(x))$.

High-concept stuff right here (Newton-Raphson)

- Given a function $f : \mathbb{R} \to \mathbb{R}$, we seek to iteratively compute the minima.
- Essentially, we want to construct a sequence $\{x_i \mid i \in \mathbb{N}\}$ which converges to $\operatorname{argmin}_{x \in \mathbb{R}} (f(x))$.
- Second-order Taylor approximation gives $f(x_k + t) \approx f(x_k) + f'(x_k)t + \frac{1}{2}f''(x_k)t^2$

High-concept stuff right here

- To make it a Cauchy sequence, we want to minimize this ”step”. Which means setting the derivative to 0.

High-concept stuff right here

- To make it a Cauchy sequence, we want to minimize this ”step”. Which means setting the derivative to 0.
- $0 = \frac{d}{dt}\left(f(x_k) + f'(x_k)t + \frac{1}{2}f''(x_k)t^2\right) = f'(x_k) + f''(x_k)t.$

High-concept stuff right here

- To make it a Cauchy sequence, we want to minimize this ”step”. Which means setting the derivative to 0.
- $0 = \frac{d}{dt}\left(f(x_k) + f'(x_k)t + \frac{1}{2}f''(x_k)t^2\right) = f'(x_k) + f''(x_k)t.$
- $x_{k+1} = x_k - \frac{f'(x_k)}{f''(x_k)}.$

High-concept stuff right here

- To make it a Cauchy sequence, we want to minimize this "step". Which means setting the derivative to 0.
- $0 = \frac{d}{dt}\left(f(x_k) + f'(x_k)t + \frac{1}{2}f''(x_k)t^2\right) = f'(x_k) + f''(x_k)t.$
- $x_{k+1} = x_k - \frac{f'(x_k)}{f''(x_k)}.$
- For $f : \mathbb{R}^n \to \mathbb{R}$, we can restate this as $\mathbf{x}_{k+1} = \mathbf{x}_k - \nabla f \cdot \mathbf{H}^{-1}$

If at first you don't succeed...

- More recent methods use linear transformations that can be parallelized like crazy (thanks, Vaswani et al!)

If at first you don't succeed...

- More recent methods use linear transformations that can be parallelized like crazy (thanks, Vaswani et al!)
- For reasons, we can try to push nonzero singular values of a matrix towards 1.

If at first you don't succeed...

- More recent methods use linear transformations that can be parallelized like crazy (thanks, Vaswani et al!)
- For reasons, we can try to push nonzero singular values of a matrix towards 1.
- As it turns out, odd polynomials can *commute* with the SVD: $p(U\Sigma V^\top) = Up(\Sigma)V^\top$

If at first you don't succeed...

- More recent methods use linear transformations that can be parallelized like crazy (thanks, Vaswani et al!)
- For reasons, we can try to push nonzero singular values of a matrix towards 1.
- As it turns out, odd polynomials can *commute* with the SVD: $p(U\Sigma V^\top) = Up(\Sigma)V^\top$
- Now, it turns out that there are some $p : \mathbb{R} \to \mathbb{R}$ where $\lim_{n\to\infty} f^n(x) = \operatorname{sgn}(x)$ at least close to 0. (Demo?)

If at first you don't succeed...

- More recent methods use linear transformations that can be parallelized like crazy (thanks, Vaswani et al!)
- For reasons, we can try to push nonzero singular values of a matrix towards 1.
- As it turns out, odd polynomials can *commute* with the SVD: $p(U\Sigma V^\top) = Up(\Sigma)V^\top$
- Now, it turns out that there are some $p : \mathbb{R} \to \mathbb{R}$ where $\lim_{n\to\infty} f^n(x) = \operatorname{sgn}(x)$ at least close to 0. (Demo?)
- So selecting the coefficients such that this holds should give us a good enough approximation.

If at first you don't succeed...

- More recent methods use linear transformations that can be parallelized like crazy (thanks, Vaswani et al!)
- For reasons, we can try to push nonzero singular values of a matrix towards 1.
- As it turns out, odd polynomials can *commute* with the SVD: $p(U\Sigma V^\top) = Up(\Sigma)V^\top$
- Now, it turns out that there are some $p : \mathbb{R} \to \mathbb{R}$ where $\lim_{n\to\infty} f^n(x) = \operatorname{sgn}(x)$ at least close to 0. (Demo?)
- So selecting the coefficients such that this holds should give us a good enough approximation.
- This is a surprise tool which will help us later.

Deep Learning

Nearly there

- Most functions we encounter are nonlinear (sorry LA folks).

Nearly there

- Most functions we encounter are nonlinear (sorry LA folks).
- We can learn to approximate some functions.

Nearly there

- Most functions we encounter are nonlinear (sorry LA folks).
- We can learn to approximate some functions.
- Indeed, Deep Learning is all about picking a parametrized function $\hat{f}(\mathbf{x}; \Theta)$ which is supposed to approximate $f(\mathbf{x})$ given the right choice of parameters Θ.

Nearly there

- Most functions we encounter are nonlinear (sorry LA folks).
- We can learn to approximate some functions.
- Indeed, Deep Learning is all about picking a parametrized function $\hat{f}(\mathbf{x}; \Theta)$ which is supposed to approximate $f(\mathbf{x})$ given the right choice of parameters Θ.
- Methods used to finally reach the right values of the parameters will be seen soon.

Classification

- We can treat an n-way classification task as a function $f : \mathbb{R}^m \to \mathbb{R}^n$ where the outputs form a probability distribution over the classes.

Classification

- We can treat an n-way classification task as a function $f : \mathbb{R}^m \to \mathbb{R}^n$ where the outputs form a probability distribution over the classes.
- We can also see it as a task of drawing $(n-1)$ $(m-1)$-hyperplanes to divide m-space into n sections, each corresponding to one class.

Classification

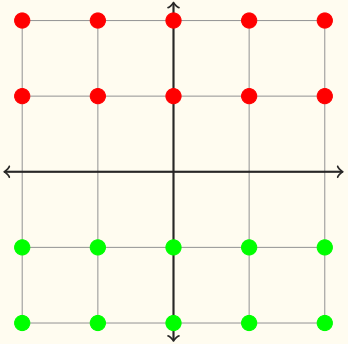

Figure: Pretty straightforward to slice

Classification

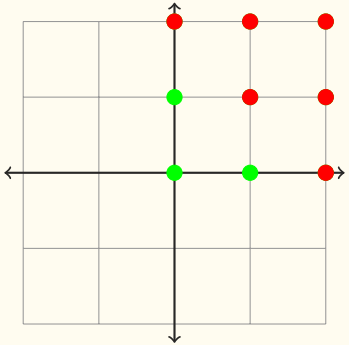

Figure: Still straightforward to slice

Classification

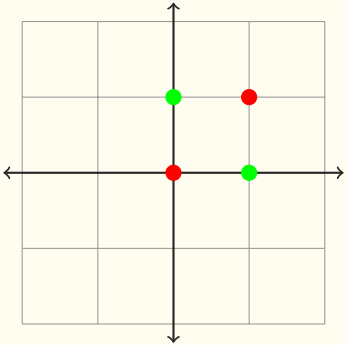

Figure: Oh no

Ain't linear no mo

- If $\mathbf{A}_1, \mathbf{A}_2, \ldots$ are matrices and $\mathbf{b}_1, \mathbf{b}_2, \ldots$ are bias vectors (all decently compatible with $\mathbf{x}$, I'm too lazy to LaTeX it all out), then consider

$$\mathbf{A}_1(\mathbf{A}_2(\cdots) + \mathbf{b}_2) + \mathbf{b}_1$$

Ain't linear no mo

- If $\mathbf{A}_1, \mathbf{A}_2, \ldots$ are matrices and $\mathbf{b}_1, \mathbf{b}_2, \ldots$ are bias vectors (all decently compatible with $\mathbf{x}$, I'm too lazy to LaTeX it all out), then consider

$$\mathbf{A}_1(\mathbf{A}_2(\cdots) + \mathbf{b}_2) + \mathbf{b}_1$$

- But this can just be written as $\mathbf{A}'\mathbf{x} + \mathbf{b}'$ for some $\mathbf{A}', \mathbf{b}'$. Affine transformations composed are just affine transformations.

Ain't linear no mo

- If $\mathbf{A}_1, \mathbf{A}_2, \ldots$ are matrices and $\mathbf{b}_1, \mathbf{b}_2, \ldots$ are bias vectors (all decently compatible with $\mathbf{x}$, I'm too lazy to LaTeX it all out), then consider

$$\mathbf{A}_1(\mathbf{A}_2(\cdots) + \mathbf{b}_2) + \mathbf{b}_1$$

- But this can just be written as $\mathbf{A}'\mathbf{x} + \mathbf{b}'$ for some $\mathbf{A}', \mathbf{b}'$. Affine transformations composed are just affine transformations.
- What happens if you introduce a nonlinearity into the mix? Try $f_i(\mathbf{x}) = f(\mathbf{A}_i\mathbf{x} + \mathbf{b}_i)$ and then $f_1(f_2(f_3(\cdots)))$.

Universal Approximation Theorem

If $\sigma : \mathbb{R} \to \mathbb{R}$ is a continuous function that can be applied piecewise, then for every $n, m \in \mathbb{N}$, continuous $f : K \to \mathbb{R}^m$ where $K \subseteq \mathbb{R}^n$ is compact, and $\epsilon > 0$, there exist $k \in \mathbb{N}$, $\mathbf{A}_1 \in \mathbb{R}^{k \times n}, \mathbf{A}_2 \in \mathbb{R}^{m \times k}, \mathbf{b} \in \mathbb{R}^k$ such that

$$\sup_{\mathbf{x} \in K} \| f(\mathbf{x}) - \mathbf{A}_2 \cdot \sigma(\mathbf{A}_1 \mathbf{x} + \mathbf{b}) \| < \epsilon$$

Universal Approximation Theorem (abridged)

Any continuous function can be approximated by a 2-layer perceptron with arbitrary precision.

Universal Approximation Theorem (proof)

Uhhhh here's a unicorn!

Begging the question 101

- So far we've been assuming that the correct parameters (weights and biases) can be chosen by some oracle. How do we get them IRL?

Begging the question 101

- So far we've been assuming that the correct parameters (weights and biases) can be chosen by some oracle. How do we get them IRL?
- We know what we want, we know what the function will give us. Find some metric to compare them by!

Begging the question 101

- So far we've been assuming that the correct parameters (weights and biases) can be chosen by some oracle. How do we get them IRL?
- We know what we want, we know what the function will give us. Find some metric to compare them by!
- The most popular now is Cross-Entropy Loss:

$$\mathcal{L}\left(\hat{f}(\mathbf{x};\Theta), f(\mathbf{x})\right) = -\sum_{i=0}^{n} y_i \log(\hat{y}_i)$$

Chain rule clutch, as always

- We've seen derivatives of scalar-valued and vector-valued functions, but Θ is not even a matrix! How do we differentiate with respect to that? How can we efficiently get the derivatives w.r.t each individual parameter?

Chain rule clutch, as always

- We've seen derivatives of scalar-valued and vector-valued functions, but Θ is not even a matrix! How do we differentiate with respect to that? How can we efficiently get the derivatives w.r.t each individual parameter?
- If there's a 1D perceptron layer $\hat{f} = f(ax + b)$, we must have that $\frac{\partial \mathcal{L}}{\partial a} = \frac{\partial \mathcal{L}}{\partial \hat{f}} \frac{\partial \hat{f}}{\partial (ax+b)} \frac{\partial (ax+b)}{\partial a}$.

Chain rule clutch, as always

- We've seen derivatives of scalar-valued and vector-valued functions, but Θ is not even a matrix! How do we differentiate with respect to that? How can we efficiently get the derivatives w.r.t each individual parameter?
- If there's a 1D perceptron layer $\hat{f} = f(ax + b)$, we must have that $\frac{\partial \mathcal{L}}{\partial a} = \frac{\partial \mathcal{L}}{\partial \hat{f}} \frac{\partial \hat{f}}{\partial (ax+b)} \frac{\partial (ax+b)}{\partial a}$.
- Similarly we can obtain $\frac{\partial \mathcal{L}}{\partial x}$ and $\frac{\partial \mathcal{L}}{\partial b}$.

Chain rule clutch, as always

- We've seen derivatives of scalar-valued and vector-valued functions, but Θ is not even a matrix! How do we differentiate with respect to that? How can we efficiently get the derivatives w.r.t each individual parameter?
- If there's a 1D perceptron layer $\hat{f} = f(ax + b)$, we must have that $\frac{\partial \mathcal{L}}{\partial a} = \frac{\partial \mathcal{L}}{\partial \hat{f}} \frac{\partial \hat{f}}{\partial (ax+b)} \frac{\partial (ax+b)}{\partial a}$.
- Similarly we can obtain $\frac{\partial \mathcal{L}}{\partial x}$ and $\frac{\partial \mathcal{L}}{\partial b}$.
- Now, if there's more than one perceptron layer, we already have $\frac{\partial \mathcal{L}}{\partial x}$, where x can be replaced by the output of the new layer. Chain rule will once again apply!

Chain rule clutch, as always

- We've seen derivatives of scalar-valued and vector-valued functions, but Θ is not even a matrix! How do we differentiate with respect to that? How can we efficiently get the derivatives w.r.t each individual parameter?
- If there's a 1D perceptron layer $\hat{f} = f(ax + b)$, we must have that $\frac{\partial \mathcal{L}}{\partial a} = \frac{\partial \mathcal{L}}{\partial \hat{f}} \frac{\partial \hat{f}}{\partial (ax+b)} \frac{\partial (ax+b)}{\partial a}$.
- Similarly we can obtain $\frac{\partial \mathcal{L}}{\partial x}$ and $\frac{\partial \mathcal{L}}{\partial b}$.
- Now, if there's more than one perceptron layer, we already have $\frac{\partial \mathcal{L}}{\partial x}$, where x can be replaced by the output of the new layer. Chain rule will once again apply!
- This also scales over perceptron dimensions (verify!)

Backpropagation

- We've seen derivatives of scalar-valued and vector-valued functions, but Θ is not even a matrix! How do we differentiate with respect to that? How can we efficiently get the derivatives w.r.t each individual parameter?

Backpropagation

- We've seen derivatives of scalar-valued and vector-valued functions, but Θ is not even a matrix! How do we differentiate with respect to that? How can we efficiently get the derivatives w.r.t each individual parameter?
- If there's a 1D perceptron layer $\hat{f} = f(ax + b)$, we must have that $\frac{\partial \mathcal{L}}{\partial a} = \frac{\partial \mathcal{L}}{\partial \hat{f}} \frac{\partial \hat{f}}{\partial (ax+b)} \frac{\partial (ax+b)}{\partial a}$.

Backpropagation

- We've seen derivatives of scalar-valued and vector-valued functions, but Θ is not even a matrix! How do we differentiate with respect to that? How can we efficiently get the derivatives w.r.t each individual parameter?
- If there's a 1D perceptron layer $\hat{f} = f(ax + b)$, we must have that $\frac{\partial \mathcal{L}}{\partial a} = \frac{\partial \mathcal{L}}{\partial \hat{f}} \frac{\partial \hat{f}}{\partial (ax+b)} \frac{\partial (ax+b)}{\partial a}$.
- Similarly we can obtain $\frac{\partial \mathcal{L}}{\partial x}$ and $\frac{\partial \mathcal{L}}{\partial b}$.

Backpropagation

- We've seen derivatives of scalar-valued and vector-valued functions, but Θ is not even a matrix! How do we differentiate with respect to that? How can we efficiently get the derivatives w.r.t each individual parameter?
- If there's a 1D perceptron layer $\hat{f} = f(ax + b)$, we must have that $\frac{\partial \mathcal{L}}{\partial a} = \frac{\partial \mathcal{L}}{\partial \hat{f}} \frac{\partial \hat{f}}{\partial (ax+b)} \frac{\partial (ax+b)}{\partial a}$.
- Similarly we can obtain $\frac{\partial \mathcal{L}}{\partial x}$ and $\frac{\partial \mathcal{L}}{\partial b}$.
- Now, if there's more than one perceptron layer, we already have $\frac{\partial \mathcal{L}}{\partial x}$, where x can be replaced by the output of the new layer. Chain rule will once again apply!

Backpropagation

- We've seen derivatives of scalar-valued and vector-valued functions, but Θ is not even a matrix! How do we differentiate with respect to that? How can we efficiently get the derivatives w.r.t each individual parameter?
- If there's a 1D perceptron layer $\hat{f} = f(ax + b)$, we must have that $\frac{\partial \mathcal{L}}{\partial a} = \frac{\partial \mathcal{L}}{\partial \hat{f}} \frac{\partial \hat{f}}{\partial (ax+b)} \frac{\partial (ax+b)}{\partial a}$.
- Similarly we can obtain $\frac{\partial \mathcal{L}}{\partial x}$ and $\frac{\partial \mathcal{L}}{\partial b}$.
- Now, if there's more than one perceptron layer, we already have $\frac{\partial \mathcal{L}}{\partial x}$, where x can be replaced by the output of the new layer. Chain rule will once again apply!
- This also scales over perceptron dimensions (verify!)

Oh yeah, it's all coming together

- Stochastic Gradient Descent attempts to minimize $\mathcal{L}$ with respect to Θ, not $\mathbf{x}$!

$$\theta_{k+1} = \theta_k - \eta \cdot \nabla_\theta \mathcal{L}$$

Oh yeah, it's all coming together

- Stochastic Gradient Descent attempts to minimize $\mathcal{L}$ with respect to Θ, not $\mathbf{x}$!

$$\theta_{k+1} = \theta_k - \eta \cdot \nabla_\theta \mathcal{L}$$

- SGD with Momentum, Adam and AdamW are better and better approaches to this same principle.

Oh yeah, it's all coming together

- Newton-Schulz also has lately been back in the spotlight because of the Muon optimizer.

Oh yeah, it's all coming together

- Newton-Schulz also has lately been back in the spotlight because of the Muon optimizer.

Algorithm 2 Muon

Require: Learning rate η, momentum μ

Initialize $B_0 \leftarrow 0$
for $t = 1, \ldots$ **do**
    Compute gradient $G_t \leftarrow \nabla_\theta \mathcal{L}_t(\theta_{t-1})$
    $B_t \leftarrow \mu B_{t-1} + G_t$
    $O_t \leftarrow \text{NewtonSchulz5}(B_t)$
    Update parameters $\theta_t \leftarrow \theta_{t-1} - \eta O_t$
end for
return θ_t

Pulling up to the function (English. The function is English)

- Most modern NLP methods treat language and communication in general as a function. Given an input prompt (which can be tokenized and embedded), the output is the correct token to be generated next. This is what autoregressive LM does.

Pulling up to the function (English. The function is English)

- Most modern NLP methods treat language and communication in general as a function. Given an input prompt (which can be tokenized and embedded), the output is the correct token to be generated next. This is what autoregressive LM does.
- But aren't we approximating functions over Euclidean spaces?

Sike! It's maths all the way down (hopefully)

- Since the output is a vector, outputting embeddings (which are assumed to be in continuous space) and discretizing them into the vocabulary is quite weak.

Sike! It's maths all the way down (hopefully)

- Since the output is a vector, outputting embeddings (which are assumed to be in continuous space) and discretizing them into the vocabulary is quite weak.
- Instead the output can be a $|V|$-dimensional vector whose entries are the probability of generating that token. Naturally we'll have to force the final output to have entries that sum to 1.

Sike! It's maths all the way down (hopefully)

- Since the output is a vector, outputting embeddings (which are assumed to be in continuous space) and discretizing them into the vocabulary is quite weak.
- Instead the output can be a $|V|$-dimensional vector whose entries are the probability of generating that token. Naturally we'll have to force the final output to have entries that sum to 1.
- We can tweak the activation function of the last layer for this. Instead of any other activation, we use

$$\mathrm{softmax}(\mathbf{x}) = \left[\frac{e^{-x_i}}{\sum_{j=0}^{d} e^{-x_j}} \right]$$

Fin